Internet of Things Protocol



Protocol

- Protocol แปลว่า กฎเกณฑ์ / ข้อตกลง / วิธีปฏิบัติที่กำหนดไว้
  - Company Protocol = กฎเกณฑ์หรือแนวปฏิบัติของบริษัท
  - Communication Protocol = กฎเกณฑ์ในการสื่อสารระหว่างอุปกรณ์
  - Network Protocol = กฎเกณฑ์ในการรับส่งข้อมูลบนเครือข่าย
  - HTTP Protocol = กฎเกณฑ์ในการสื่อสารระหว่าง Web Client และ Web Server โดยสามารถรับ-ส่งข้อมูลได้หลายรูปแบบ เช่น HTML, รูปภาพ, Video, CSS, JavaScript, JSON
  - IoT Protocol คือ กฎเกณฑ์หรือมาตรฐานที่กำหนดว่าอุปกรณ์ IoT จะสื่อสารและรับ–ส่งข้อมูลกันอย่างไร
    - ตัวอย่างเช่น MQTT , CoAP , AMQP

MQTT Protocol

- **MQTT : Message Queuing Telemetry Transport**
- Invented by Andy Stanford Clark (IBM) and Arlen Nipper (Eurotech) in 1999
- Originally envisioned for use over Satellite link from an oil pipeline

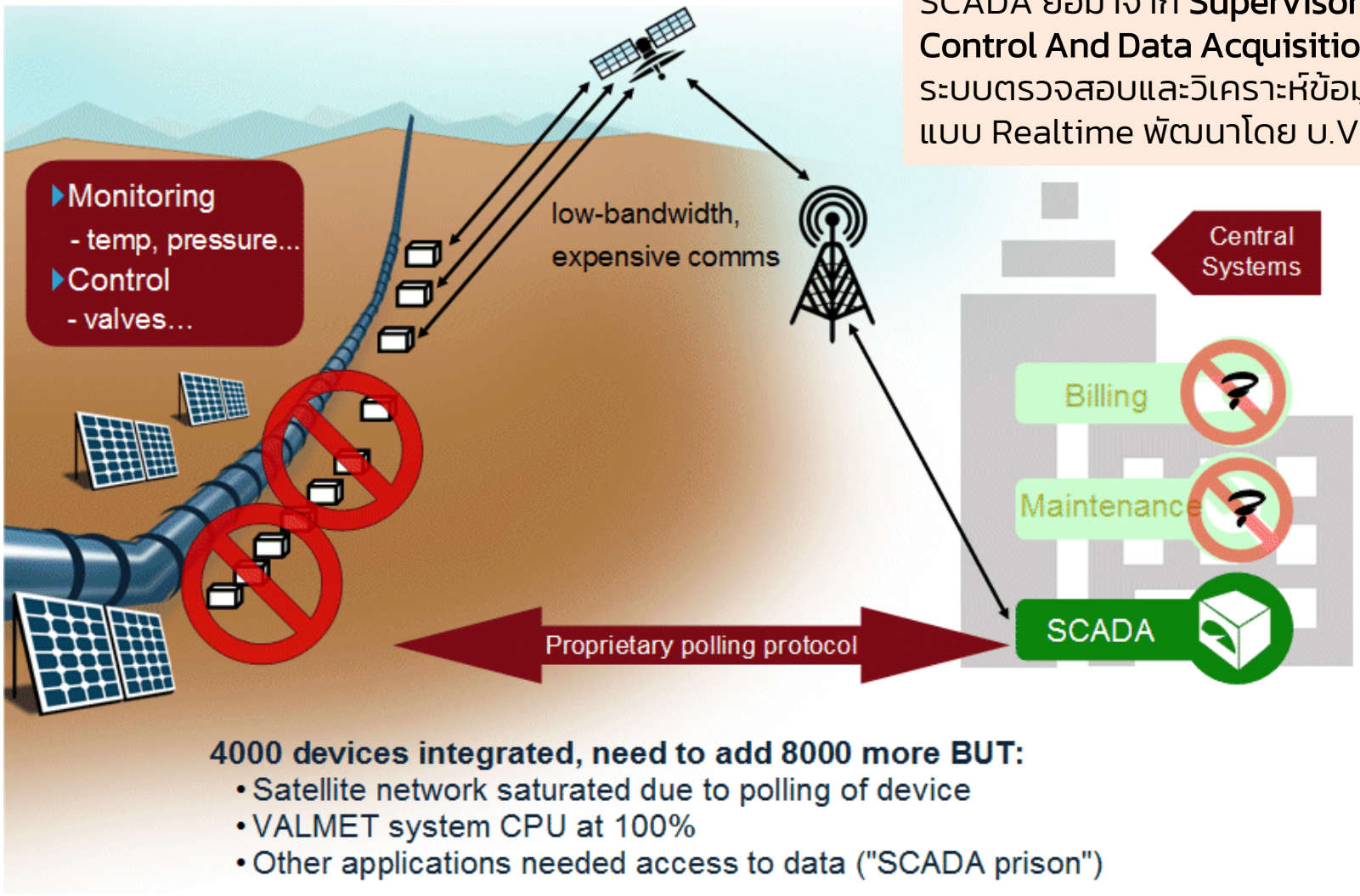


Use case:

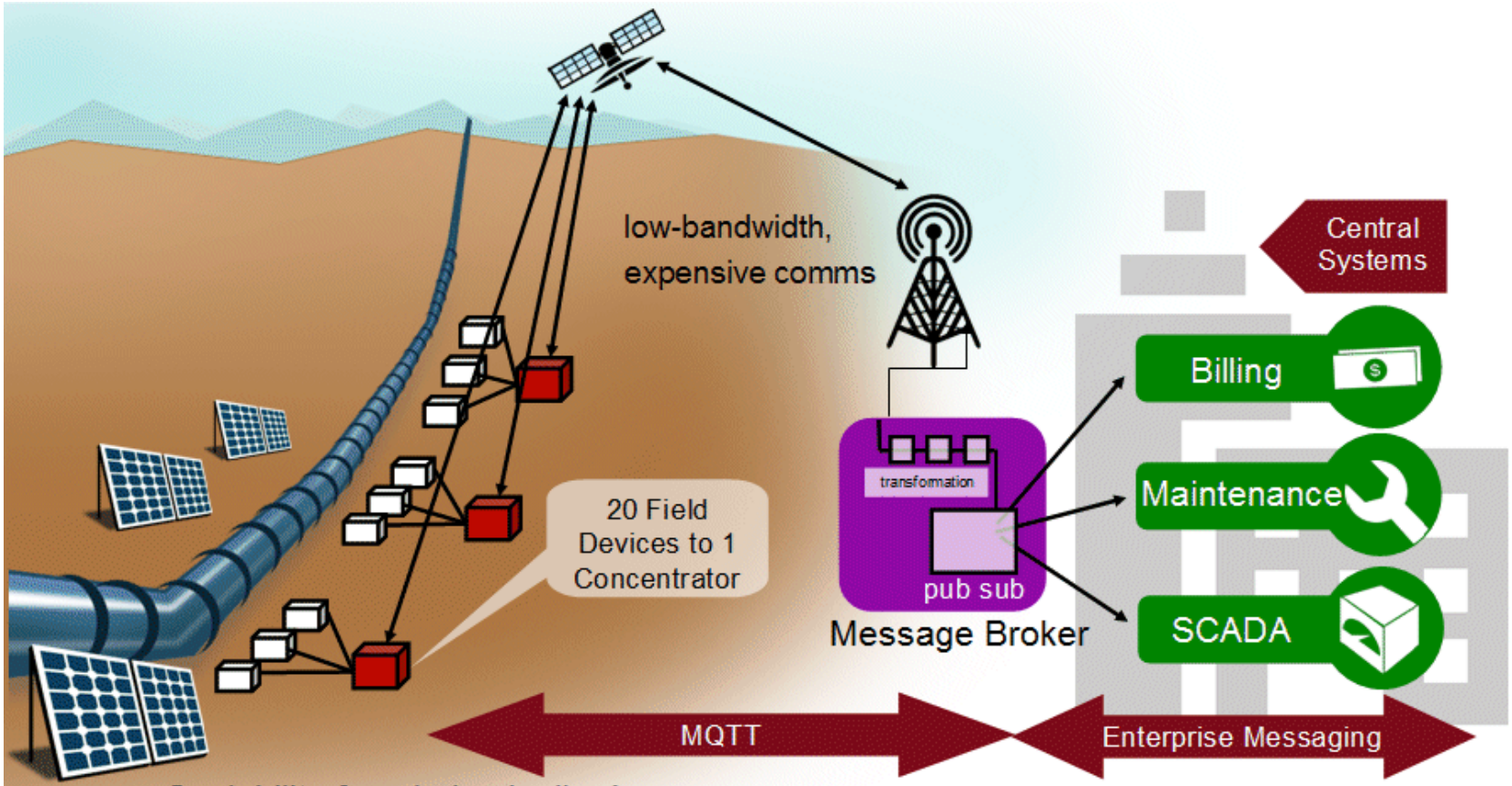
- 30.000 devices
- 17.000 km pipeline
- Remote monitoring
- Remote control
- Uses satellite links
- Bandwidth is **very** expensive

ที่มา : MQTT Protocol

SCADA ย่อมาจาก **Supervisory Control And Data Acquisition** เป็นระบบตรวจสอบและวิเคราะห์ข้อมูลแบบ Realtime พัฒนาโดย บ.Valmet



ที่มา : MQTT Protocol

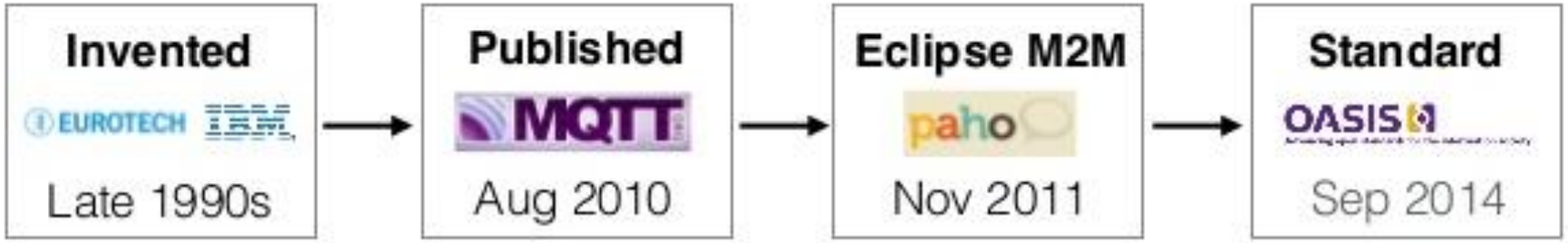


Scalability for whole pipeline!
Network traffic much lower - events pushed to/from devices and report by exception
Network cost reduced
Lower CPU utilization
Broken out of the SCADA prison – data accessible to other applications

ลักษณะของ MQTT Protocol

MQTT a lightweight protocol for IoT messaging

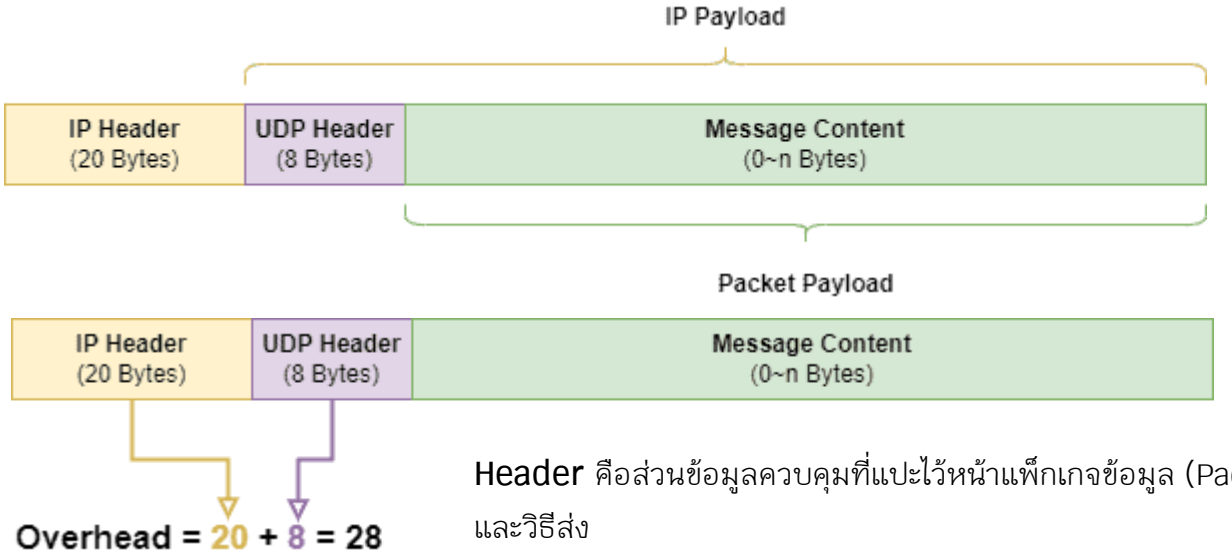
- **open** open spec, standard 40+ client implementations
- **lightweight** minimal overhead efficient format tiny clients (kb)
- **reliable** QoS for reliability on unreliable networks
- **simple** 43-page spec connect + publish + subscribe



ลักษณะของ MQTT Protocol

- **MQTT** มี Header ขนาดเล็ก (ขั้นต่ำ 2 bytes) จึงมี Protocol Overhead ต่ำ ทำให้เหมาะกับอุปกรณ์ IoT ที่มี Bandwidth และทรัพยากรจำกัด และใช้ **Persistent TCP Connection** (เปิด TCP connection ค้างไว้ กับ MQTT Broker เพื่อส่ง/รับข้อความต่อเนื่อง)

- Note!! ข้อแตกต่าง Header & Overhaed

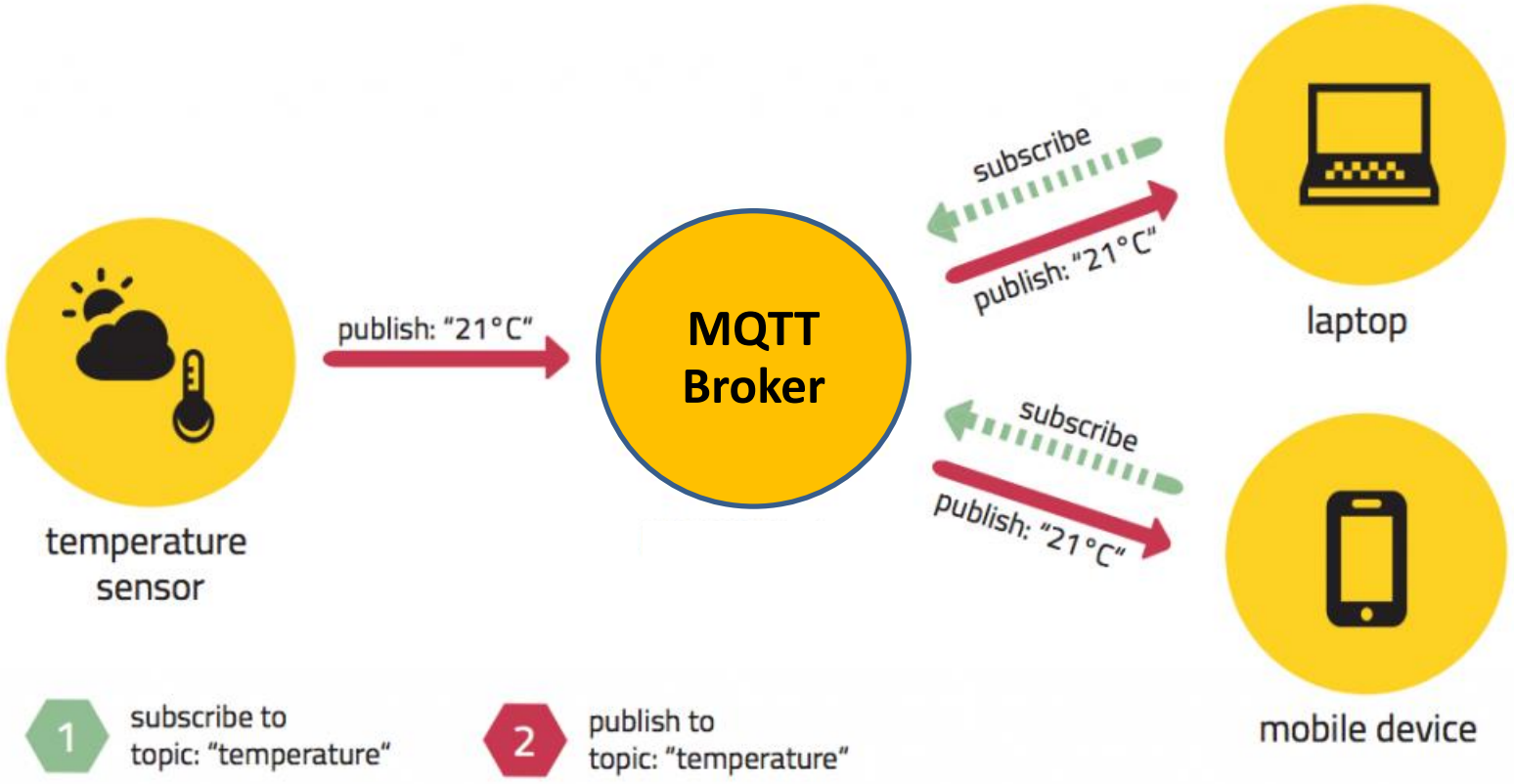


Header คือส่วนข้อมูลควบคุมที่แปะไว้หน้าแพ็กเกจข้อมูล (Packet) เพื่อบอกเส้นทางและวิธีส่ง

Overhead คือปริมาณข้อมูลส่วนเกินทั้งหมด (ซึ่งรวม Header และ Trailer) ที่ไม่ใช่ข้อมูลจริงของผู้ใช้ (Payload)

The publish/subscribe pattern

Bi-directional , Asynchronous "**Push**" Communication



MQTT Protocol

MQTT

simple to implement

Connect

Subscribe

Publish

Unsubscribe

Disconnect

```
client = new Messaging.Client(hostname, port, clientId)
client.onMessageArrived = messageArrived;
client.onConnectionLost = connectionLost;
client.connect({ onSuccess: connectionSuccess });

function connectionSuccess() {
     client.subscribe("planets/earth");
     var msg = new Messaging.Message("Hello world!");
     msg.destinationName = "planets/earth";
     client.publish(msg);
}

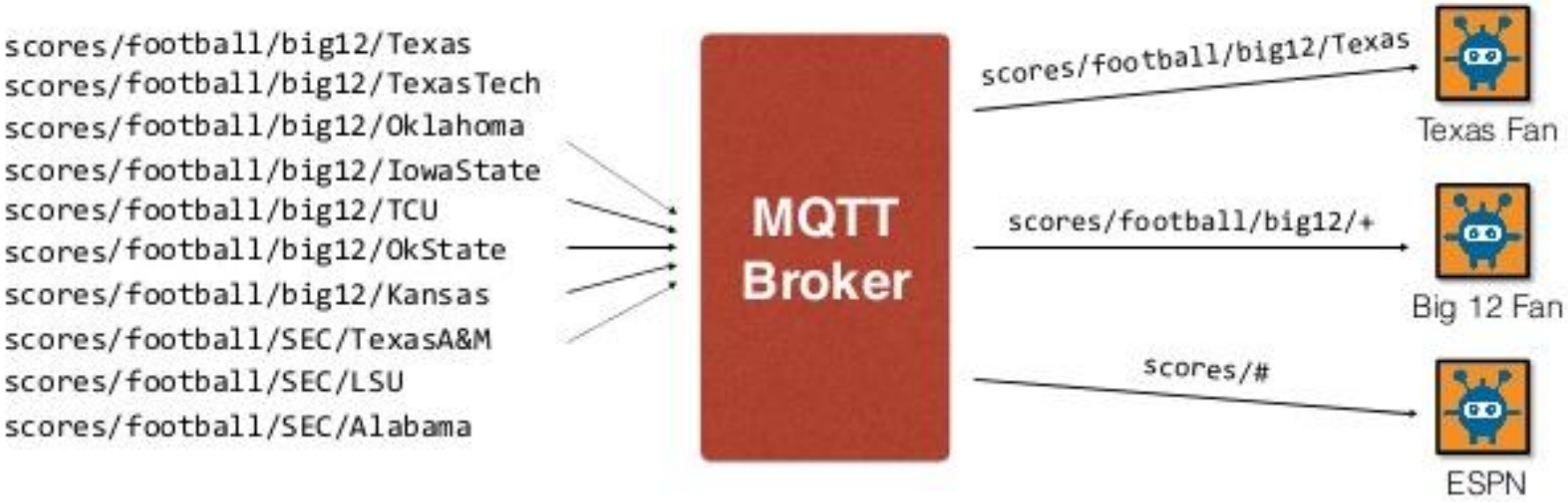
function messageArrived(msg) {
     console.log(msg.payloadString);
     client.unsubscribe("planets/earth");
     client.disconnect();
}
```

Eclipse Paho JavaScript MQTT client

Subscribe

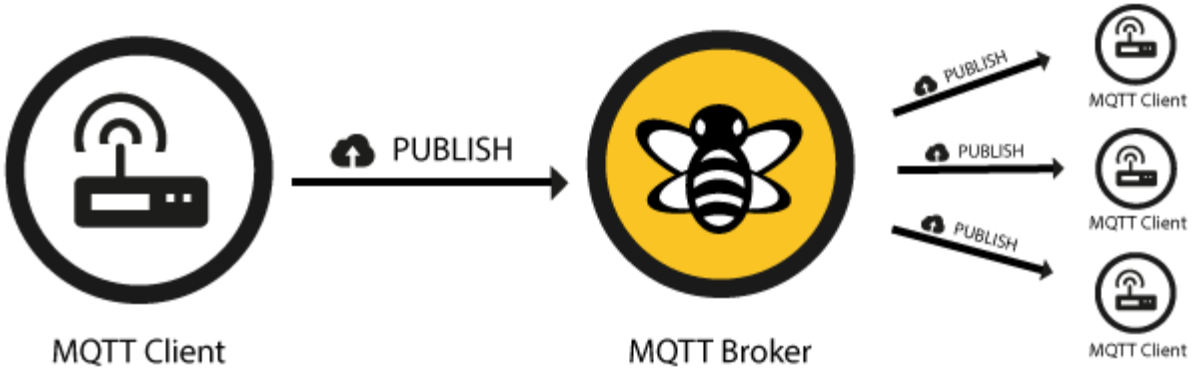
MQTT

allows wildcard subscriptions



single level wildcard: **+** multi-level wildcard: **#**

Publish



MQTT-Packet:

PUBLISH

contains:	Example
`packetId` (always 0 for qos 0)	`4314`
`topicName`	`"topic/1"`
`qos`	`1`
`retainFlag`	`false`
`payload`	`"temperature:32.5"`
`dupFlag`	`false`

Publish

Topics
A topic is a UTF-8 string, which is used by the broker to filter messages for each connected client. A topic consists of one or more topic levels. Each topic level is separated by a forward slash (topic level separator).



QoS
A Quality of Service Level (QoS) for this message. The level (0,1 or 2) determines the guarantee of a message reaching the other end

Retain-Flag
This flag determines if the message will be saved by the broker for the specified topic as last known good value. New clients that subscribe to that topic will receive the last retained message on that topic instantly after subscribing. (ให้ Broker เก็บ Payload ล่าสุด ที่ส่งขึ้นไปไว้ด้วย ถ้ามีคนใหม่เข้ามา Subscribe ทีหลัง ก็จะได้รับข้อมูลนี้ด้วย)

Payload
This is the actual content of the message.

DUP flag
The duplicate flag indicates, that this message is a duplicate and is resent because the other end didn't acknowledge the original message. This is only relevant for QoS greater than 0

Packet Identifier
The packet identifier is a unique identifier between client and broker to identify a message in a message flow. This is only relevant for QoS greater than zero.

QoS (Quality of Service)

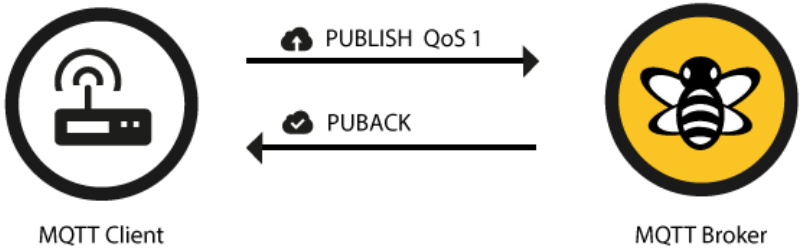
- **QoS 0 – at most once**

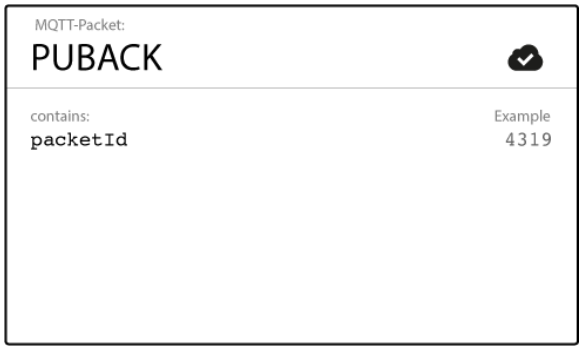
  The minimal level is zero and it guarantees a best effort delivery. A message won't be acknowledged by the receiver or stored and redelivered by the sender.



- **QoS 1 – at least once**

  it is guaranteed that a message will be delivered at least once to the receiver. The sender will store the message until it gets an acknowledgement in form of a PUBACK command message from the receiver.

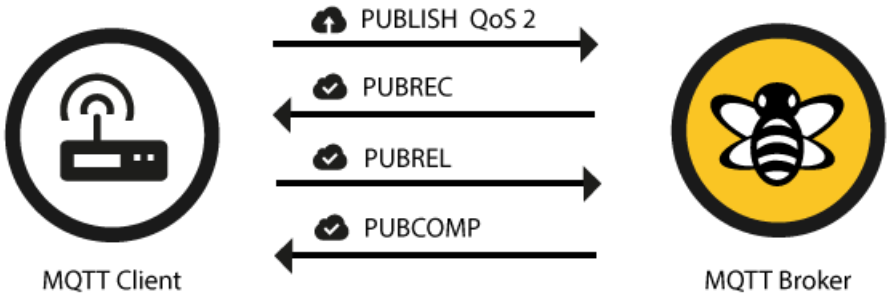




QoS

- **QoS 2 – Exactly once**

  The highest QoS is 2, it guarantees that each message is received only once by the counterpart. It is the safest and also the slowest quality of service level. The guarantee is provided by two flows there and back between sender and receiver.



MQTT-Packet:
PUBREC

contains: `packetId`
Example: 4320

Publish received

MQTT-Packet:
PUBREL

contains: `packetId`
Example: 4320

Publish release

MQTT-Packet:
PUBCOMP

contains: `packetId`
Example: 4320

Publish complete

QoS 0

- **Qos 0 :** ส่งไม่เกิน 1 ครั้ง (At most once) ไม่รับประกันว่าข้อความจะถึงปลายทาง
- **หลักการ**
  - ส่งครั้งเดียว
  - ไม่มีการยืนยันการรับ
- **ลักษณะ**
  - ข้อความอาจสูญหาย
  - ไม่มีการส่งซ้ำ
- **ขั้นตอน**



- **ข้อดี**
  - latency (เวลาในการสื่อสาร) ต่ำ เนื่องจาก Overhaed ต่ำ
  - ประหยัด bandwidth น้อย

QoS 1

- **QoS 1** : รับประกันว่าข้อความจะถูกส่ง **อย่างน้อยหนึ่งครั้ง** แต่มีโอกาสเกิด duplicate message (ข้อความซ้ำ)
- **หลักการ**
  - ต้องได้รับอย่างน้อย 1 ครั้ง
  - ถ้าไม่ได้รับ ACK จะส่งใหม่
- **ขั้นตอน**

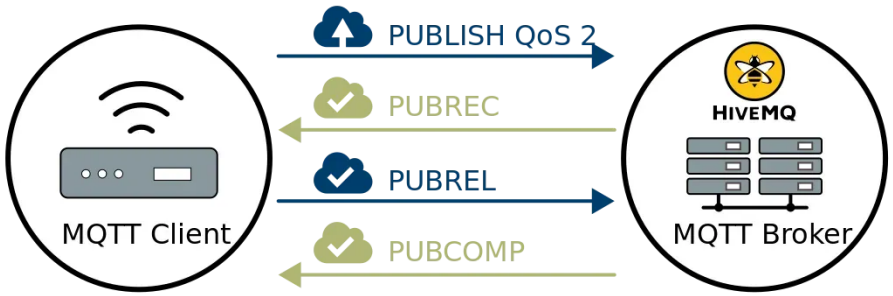


PUBACK = Publish Acknowledgement
“ยืนยันการรับข้อความ”

- **ผลที่อาจเกิด**
  - ข้อความ (message) อาจ **ซ้ำ**
- **ข้อดี**
  - reliability สูงกว่า QoS 0

QoS 2

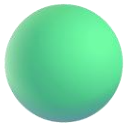
- **QoS 2 :** รับประกันว่าข้อความจะถูกส่ง “เพียงครั้งเดียว” (Exactly once) และ ไม่เกิด Duplicate Message (ข้อความซ้ำ)
- **หลักการ**
  - รับประกัน **ส่งเพียงครั้งเดียว**
- **ขั้นตอน (มี 4 ขั้นตอน)**



PUBREC = Publish Received
“ฉันได้รับข้อความแล้ว”
PUBREL = Publish Release
“ตกลง ให้ดำเนินการกับข้อความนี้ได้”
PUBCOMP = Publish Complete
“ดำเนินการเสร็จสมบูรณ์แล้ว”

- **ข้อดี**
  - ไม่มี duplicate
- **ข้อเสีย**
  - latency (เวลาในการสื่อสาร) สูงกว่า **QoS 0 และ QoS 1**
  - network overhead สูง

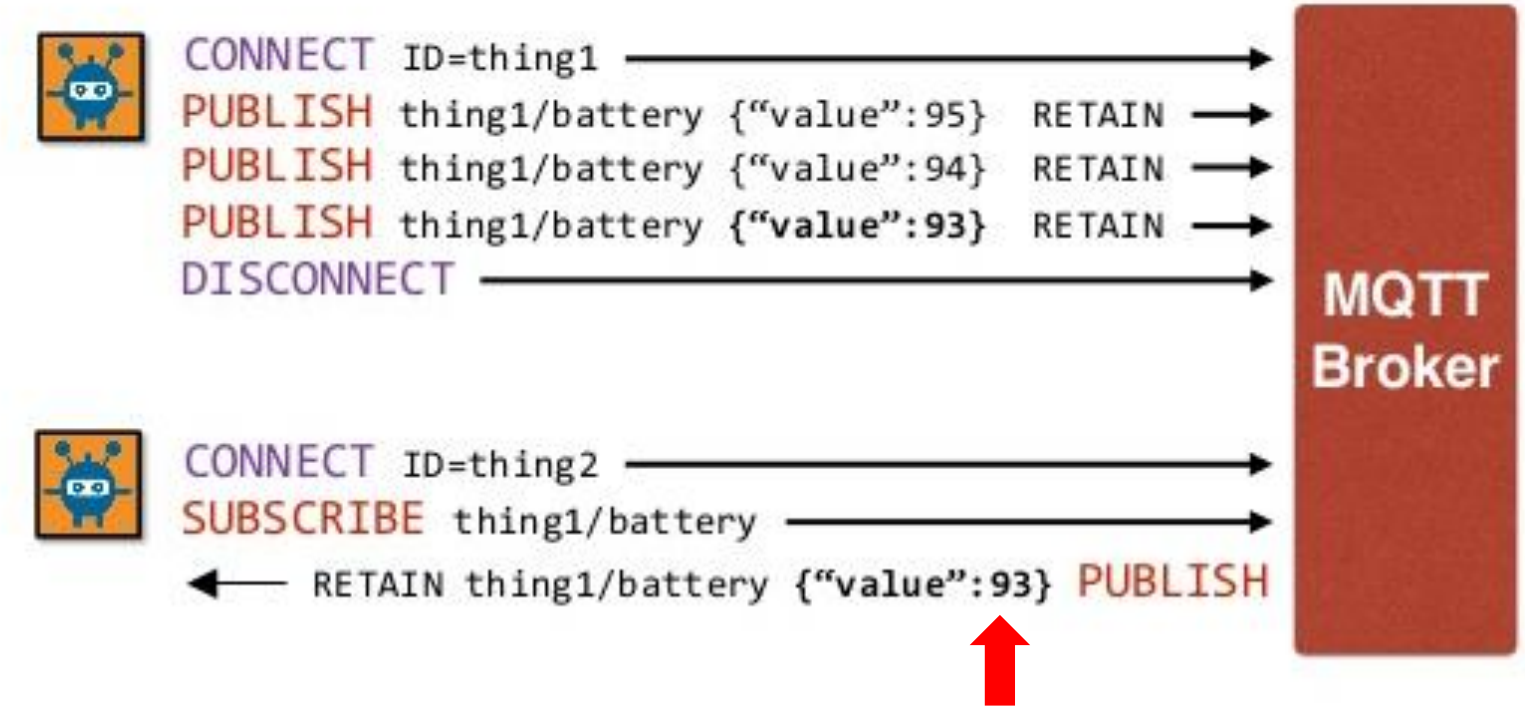
สรุป

-  **QoS 0:** ส่งครั้งเดียว ข้อความอาจ**หายได้**
   **QoS 1:** ส่งจนกว่าจะได้รับ ACK ข้อความอาจ**ซ้ำได้**
   **QoS 2:** มีกระบวนการยืนยันหลายขั้น ข้อความ**ไม่หายและไม่ซ้ำ**

- Effective QoS = **min**(Publish QoS, Subscribe QoS)

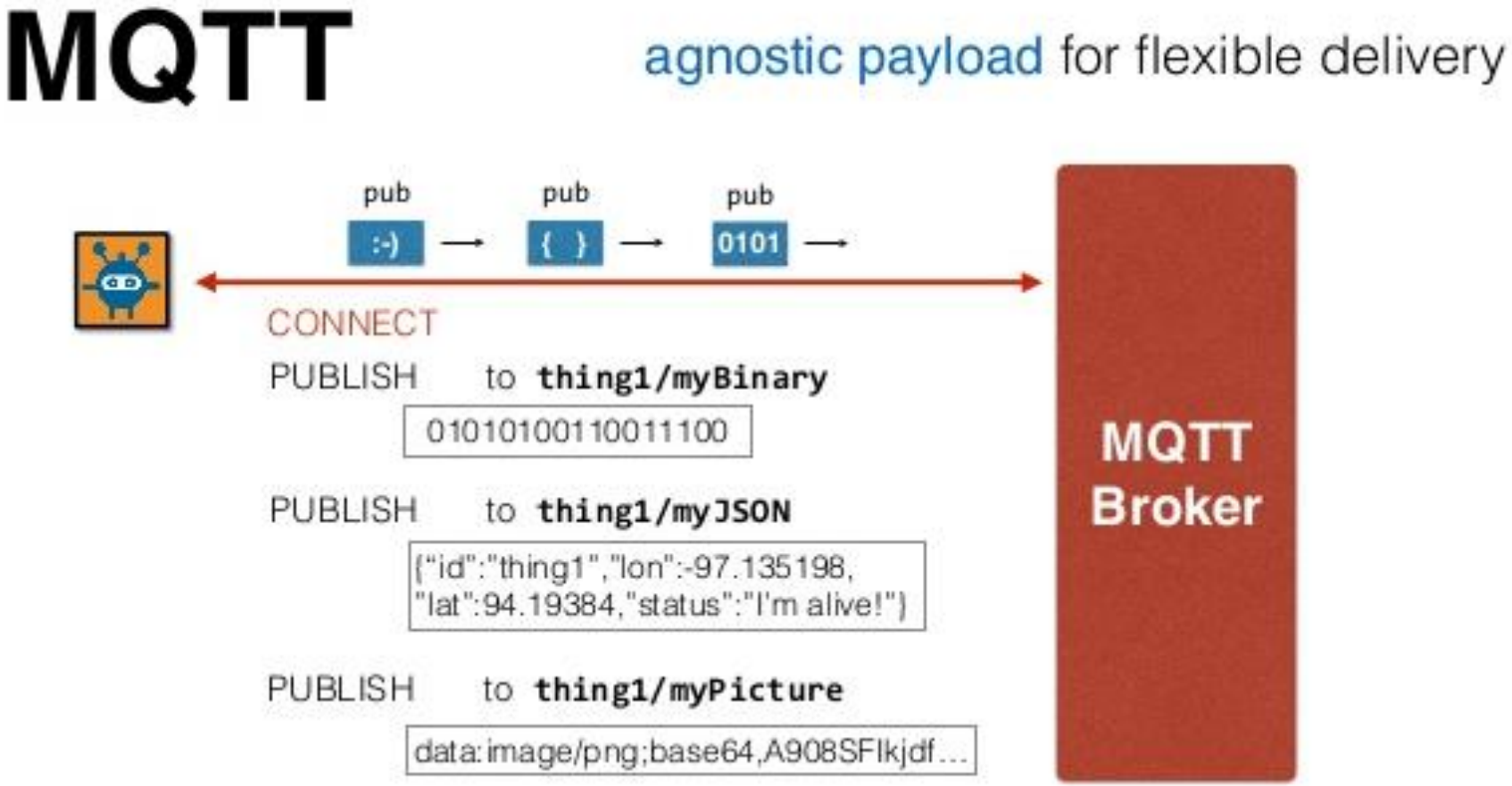
Retain messages

MQTT retained messages for last value caching



- Broker เก็บ message ล่าสุดของ topic
- Subscriber ใหม่จะได้รับข้อความนั้นทันที

Payload ส่งเป็น array

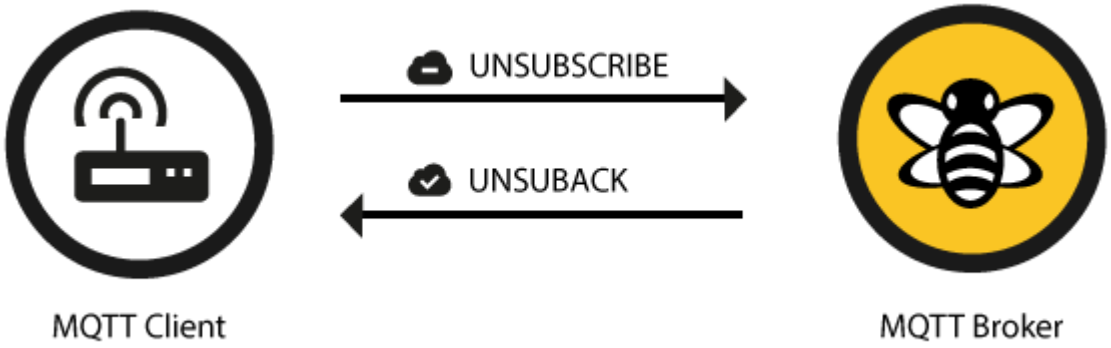


Agnostic แปลว่า ไม่ยึดติดหรือไม่ขึ้นกับสิ่งใดสิ่งหนึ่ง

Payload

- ในโปรโตคอล MQTT ข้อมูลที่ส่งใน **Payload** ไม่มีการกำหนดชนิดข้อมูลตายตัว (data type agnostic) โดย payload ถูกส่งในรูปแบบ **array of bytes (ชุดข้อมูลของ byte)** ดังนั้นจึงสามารถส่งข้อมูลได้หลายรูปแบบ เช่น
    - String
    - JSON
    - XML
    - ตัวเลข
    - รูปภาพ
    - ไฟล์

Unsubscribe



MQTT-Packet:

UNSUBSCRIBE

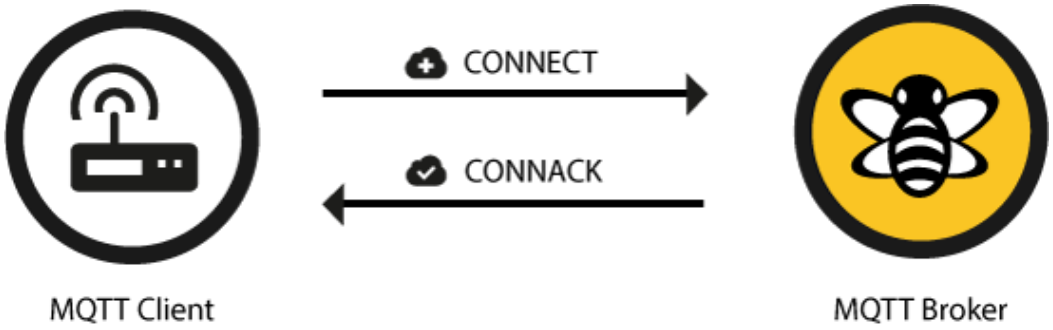
contains:		Example
`packetId`		`4315`
`topic1`	(list of topics)	`"topic/1"`
`topic2`		`"topic/2"`
`...`		`...`

MQTT-Packet:

UNSUBACK

contains:	Example
`packetId`	`4316`

Connect & Disconnect



MQTT-Packet:

CONNECT

contains:	Example
clientId	"client-1"
cleanSession	true
username (optional)	"hans"
password (optional)	"letmein"
lastWillTopic (optional)	"/hans/will"
lastWillQos (optional)	2
lastWillMessage (optional)	"unexpected exit"
keepAlive	60

MQTT-Packet:

CONNACK

contains:	Example
sessionPresent	true
returnCode	0

MQTT-Packet:

DISCONNECT

no payload

Client ID จะต้องไม่ซ้ำกัน
ถ้ามี ID ซ้ำกัน อุปกรณ์ที่อยู่
ก่อน จะโดน Broker เตะออก

Feature ของ MQTT
1. Last Will and Testament

- Last Will แปลว่า พินัยกรรมฉบับสุดท้าย” หรือ “ความประสงค์ครั้งสุดท้าย
- testament แปลว่า “เป็นหลักฐาน/เป็นสิ่งยืนยันถึง...”

ดังนั้น

- Last Will and Testament แปลว่า ข้อความที่ฝากไว้ก่อนตาย/ก่อนขาดการติดต่อ หรือ ข้อความสุดท้ายที่ฝากไว้ หากหายไปโดยไม่ได้บอกลา
- LWT เป็น Feature มาตรฐานของ MQTT

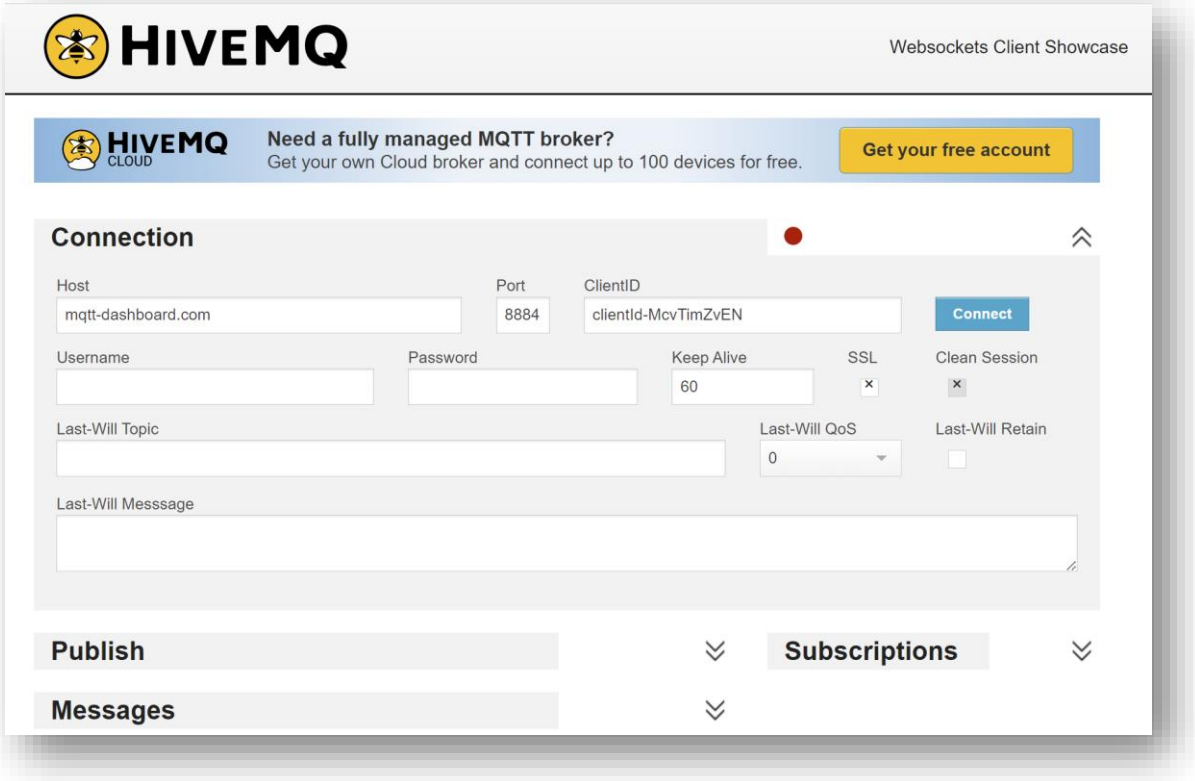
Feature ของ MQTT
1. Last Will and Testament

- หลักการของ Last Will and Testament (LWT)
  - LWT คือ ข้อความที่ broker จะส่งออกอัตโนมัติ เมื่อ client หลุดการเชื่อมต่อแบบผิดปกติ เช่น ไฟดับ , network หลุด , โปรแกรม crash , ไม่ส่ง keep-alive ตามเวลาที่กำหนด
  - เมื่อเกิดเหตุการณ์ต่าง ๆ เหล่านี้ Broker จะทำการ publish ข้อความ LWT ไปยัง Topic ที่กำหนดไว้

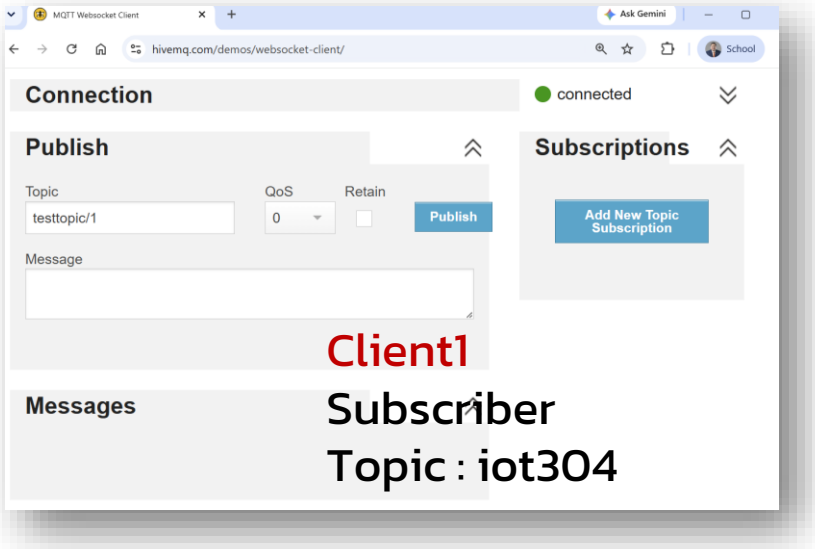
การทดลองที่ 1.1 Public MQTT Broker

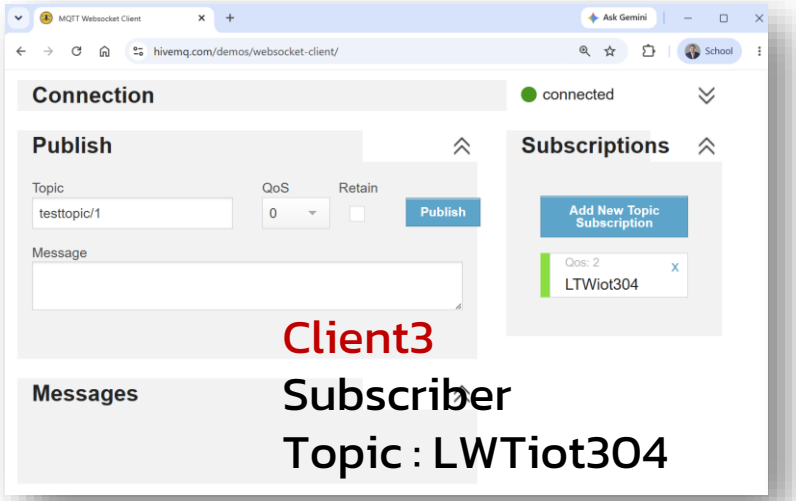
- เปิด web browser เข้า Website ที่ URL
  http://www.hivemq.com/demos/websocket-client/

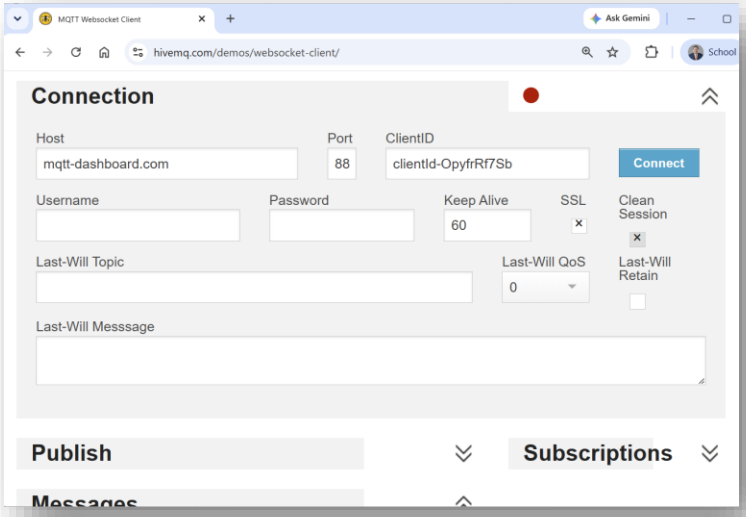
  จำนวน 3 หน้าจอ



การทดลองที่ 1.1 LWT







Client2
Publisher
ตั้งค่า LWT-Topic : LTWiot304
ตั้งค่า LWT-Message : Alert!!!

ทำการ Publish ไปยัง iot304
จากนั้นทำการปิด Browser

การทดลองที่ 1.1 LWT

- เปิด web browser จำนวน 3 หน้าจอ
  - Client 1 (Subscriber)
    - ทำการ Connect ไปยัง Broker
    - ทำการ Subscribe ไปยัง Topic ใดๆ ที่กำหนด เช่น Topic : IoT304
  - Client 2 (Publisher)
    - ตั้งค่า LWT (Last-Will Topic, Last-Will Message)
      - เช่น Last-Will Topic : LWTiot304 , Last-Will Message : Alert !!!
    - ทำการ Connect ไปยัง Broker
    - ทำการลอง Publish ไปยัง Topic ใดๆ ที่กำหนด เช่น Topic : IoT304
    - สังเกตว่า จอที่ 1 ได้รับข้อความ

การทดลองที่ 1.1 LWT

- Client 3 (LWT Subscriber)
  - ทำการ Connect ไปยัง Broker
  - ทำการ Subscribe ไปยัง Topic ของ LWT
    - ในที่นี้ ทำการ subscribe Last-Will Topic : LTWiot304

- จากนั้นทำการ ปิด Browser ของ Client 2 หรือ Refresh (ห้ามกด Disconnect)
- ผลลัพธ์ที่ได้ : Client 3 จะได้รับข้อความอัตโนมัติว่า Alert!!!

Feature ของ MQTT

2. Session Management (Stateful Connection)

- MQTT สามารถรักษา “ Session ของ Client” ไว้ที่ Broker แม้ Client จะหลุดการเชื่อมต่อชั่วคราว
- ปัญหาของระบบ IoT คือ Client อาจ Offline ได้ตลอดเวลา เช่น

  Wi-Fi หลุด , Internet ขัดข้อง
- ถ้า Subscriber มีการเปิดใช้ Session เมื่อ Subscriber กลับมาเชื่อมต่อ ก็จะได้รับข้อมูลที่หายไป ในช่วงที่ขาดการเชื่อมต่อ
- Session Management ใช้ได้กับ MQTT Version 5

Feature ของ MQTT

2. Session Management (Stateful Connection)

- พารามิเตอร์หลักที่ต้องใช้ คือ
  - Client ID ต้องมีค่าเดิม เพื่อทราบว่า อุปกรณ์เดิมได้กลับมา
  - Clean Start ต้องมีค่าเป็น False (โดยค่า default จะมีค่าเป็น True ซึ่งหมายถึงให้เปิด Session ใหม่ ทุกครั้งที่มีการเชื่อมต่อ)
  - Session Expiry Interval คือ การกำหนดว่า Broker จะเก็บ Session ไว้นานเท่าไร หลังจาก Client Disconnect

Feature ของ MQTT
3. Keep Alive

- Keep Alive **คือ** ค่าที่กำหนดให้ client ต้องส่ง packet ไปยัง broker ภายในเวลาที่กำหนด (วินาที) เพื่อรักษา **MQTT connection** และตรวจสอบว่า connection ยัง active
- Keep Alive เหมือนเป็นการตรวจสอบว่าอีกฝ่ายยังออนไลน์อยู่หรือไม่ ?
- Client ต้องการบอก Broker ว่า ถ้าไม่มี MQTT Control Packet ใด ๆ ส่งมา ฉันจะส่ง PING Request เพื่อบอกว่า Client ยังมีชีวิตอยู่

- ถ้า broker ไม่ได้รับ packet ภายในเวลาที่กำหนด
  - 1.5 × Keep Alive
  - broker จะถือว่า Client **Disconnect** และอาจ trigger **Last Will and Testament (LWT)**

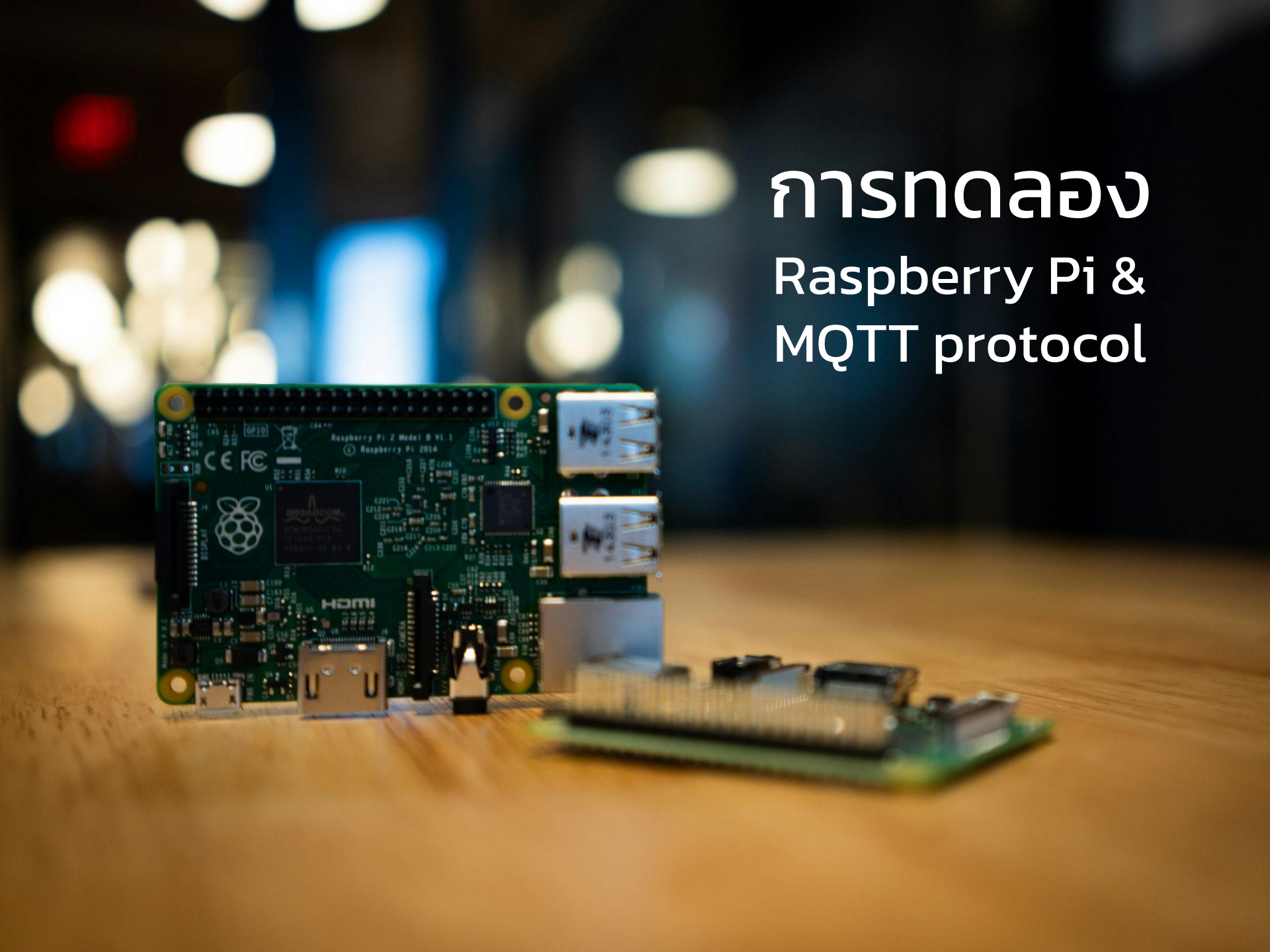
Feature ของ MQTT
3. Keep Alive

หลักการทำงานของ Keep Alive

- เมื่อ Client เชื่อมต่อกับ broker
- Client กำหนดค่า **Keep Alive** เช่น Keep Alive = 60 seconds
- ถ้าในช่วง **60 วินาที** ไม่มีการส่ง packet ใด ๆ เช่น PUBLISH , SUBSCRIBE , PUBACK
  - Client จะส่ง packet “PINGREQ” ไปยัง broker
  - Broker จะตอบกลับ “PINGRESP” เพื่อยืนยันว่า connection ยังใช้งานได้

การทดลอง

Raspberry Pi & MQTT protocol



การติดตั้ง Library สำหรับภาษา Python ให้ใช้งานโปรโตคอล MQTT ได้

พิมพ์คำสั่งในการติดตั้งดังนี้

```
$sudo apt install python3-paho-mqtt
```

รายละเอียด

https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html

การทดลองที่ 1. Publish/Subscribe โดย Raspi

Publisher





Broker : mqtt-dashboard.com

Subscriber



1. **Publisher :** ทำการ publish ไปเรื่อย ๆ เช่น ทุก 2 วินาที
2. **Subscriber :** ทำการรอรับไปเรื่อย ๆ

Publisher : ใช้ code pub.py

Subscriber : ใช้ code sub.py

ตัวอย่าง MQTT Publish โดยภาษา python

- พิมพ์คำสั่ง **$nano pub.py**
- เขียนโปรแกรม python ดังนี้

```
import  paho.mqtt.client as mqtt
import  time
mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
mqttc.connect("mqtt-dashboard.com", 1883)
while True:
        mqttc.publish("test/pub", "Hello" , qos=1)
        time.sleep(2)
```

- Run โปรแกรมโดยใช้คำสั่ง **$python3 pub.py**

ตัวอย่าง MQTT Subscribe โดยภาษา python

- พิมพ์คำสั่ง **$nano sub.py**
- เขียนโปรแกรม python ดังนี้

```
import paho.mqtt.client as mqtt

def on_message(client, userdata, msg):
    print("client =", client)
    print("userdata =", userdata)
    print("topic =", msg.topic)
    print("payload =", msg.payload.decode())
    print("QoS     :", msg.qos)
    print("Retain  :", msg.retain)

mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
mqttc.on_message = on_message
mqttc.connect("mqtt-dashboard.com", 1883)
mqttc.subscribe("test/pub", qos=1)
mqttc.loop_forever()
```

- Run โปรแกรมโดยใช้คำสั่ง **$python3 sub.py**

การทดลองที่ 2. LWT โดย Raspi

Publisher



Publisher : ใช้ code pub_lwt.py

Broker :
mqtt-dashboard.com

Subscriber

LWT Subscriber

1. **Publisher :**
    1. ตั้งค่า LWT ใน code ที่กำหนดให้
    2. ทำการ run โปรแกรม python บน Raspberry pi
       โปรแกรมทำการ publish ไปเรื่อย ๆ เช่น ทุก 2 วินาที
2. **Subscriber :** ใช้ web browser ทำการ subscribe topic ที่ publisher ส่งมา
3. **LWT Subscriber :** ใช้ web browser ทำการ subscribe topic ที่ publisher ได้ตั้งค่า LWT ไว้
4. ลองตัด internet ของ Raspberry pi แล้วดูว่า LWT Subscriber ได้รับค่า LWT message หรือไม่ ?

ตัวอย่าง MQTT Publish + LWT โดยภาษา python

- พิมพ์คำสั่ง **$nano pub_lwt.py**
- เขียนโปรแกรม python ดังนี้

ในทางปฏิบัติ LTW **มักใช้กับ Publisher** เนื่องจากต้องการบอกว่าอุปกรณ์ Online/Offline

```
import  paho.mqtt.client as mqtt
import  time
mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

mqttc.will_set(
"test/status",
    payload="offline",
    qos=1,
    retain=True
)

mqttc.connect("mqtt-dashboard.com", 1883)
while True:
        mqttc.publish("test/pub", "Hello World",qos=1)
        time.sleep(2)
```

- Run โปรแกรมโดยใช้คำสั่ง **$python3 pub_lwt.py**

การทดลองที่ 3. Session Management

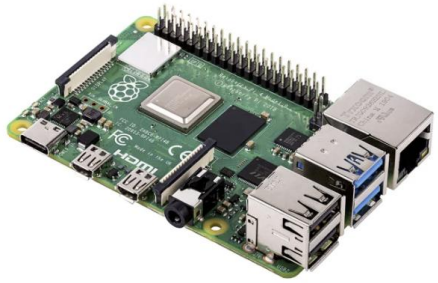
Publisher





Broker : mqtt-dashboard.com

Subscriber



1. Publisher : ทำการ publish ไปเรื่อย ๆ เช่น ทุก 2 วินาที
2. Subscriber : ทำการรอรับไปเรื่อย ๆ
3. จากนั้น ปิดการเชื่อมต่อของ Subscriber (จำลองว่าหลุดไป) เช่น Ctrl C
4. จากนั้น ทำการ connect เข้าไปใหม่
   - สังเกตว่า จะได้รับข้อมูลในช่วงที่หายไป

Publisher : ใช้ code pub_session.py

Subscriber : ใช้ code sub_session.py

ตัวอย่าง MQTT Subscribe + Session management โดยภาษา python

- พิมพ์คำสั่ง **$nano sub_session.py** เขียนโปรแกรม python ดังนี้

```
import paho.mqtt.client as mqtt

def on_message(client, userdata, msg):
    print("topic =", msg.topic)
    print("payload =", msg.payload.decode())


mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2
    client_id="panwit-sub" ,
    protocol=mqtt.MQTTv5)
mqttc.on_message = on_message

properties = mqtt.Properties(mqtt.PacketTypes.CONNECT)
properties.SessionExpiryInterval = 3600

mqttc.connect("mqtt-dashboard.com", 1883
    clean_start=False ,properties=properties)
mqttc.subscribe("test/pub", qos=1)
mqttc.loop_forever()
```

- Run โปรแกรมโดยใช้คำสั่ง **$python3 sub_session.py**

ตัวอย่าง MQTT Publish ด้วย MQTT V5 โดยภาษา python

- พิมพ์คำสั่ง **$nano pub_session.py** เขียนโปรแกรม python ดังนี้

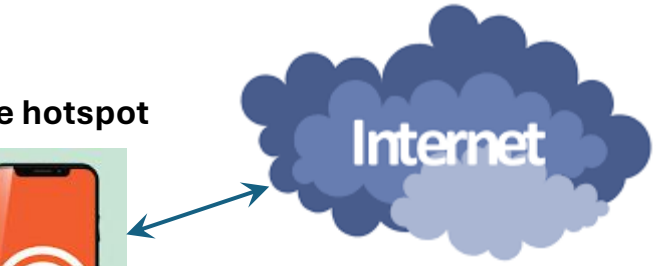
```
import paho.mqtt.client as mqtt
import  time
mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2 ,
    protocol=mqtt.MQTTv5)

mqttc.connect("mqtt-dashboard.com", 1883)

index = 1
while True:
    mqttc.publish("test/pub", "Hello "+ str(index) ,qos=1)
    time.sleep(2)
    index = index+1
```

- Run โปรแกรมโดยใช้คำสั่ง **$python3 pub_session.py**

Lab : Network Topology





mqtt-dashboard.com

หรือ

phycom.it.kmitl.ac.th

Port

mqtt : 1883

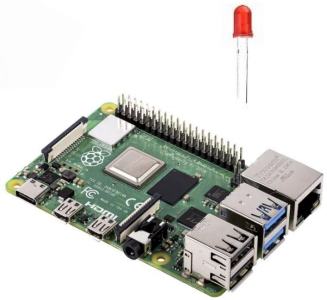
web socket : 8884

Mobile hotspot





Raspberry Pi
Publisher



Raspberry Pi
Subscriber

ต่อ ตัวต้านทาน 220 โอห์ม

และต่อหลอด LED



https://www.hivemq.com/demos/websocket-client/

เปิด web browser จำนวน 2 หน้าจอ

- **LWT Subscriber**
- **Publisher**

แบบฝึกหัดที่ 1.

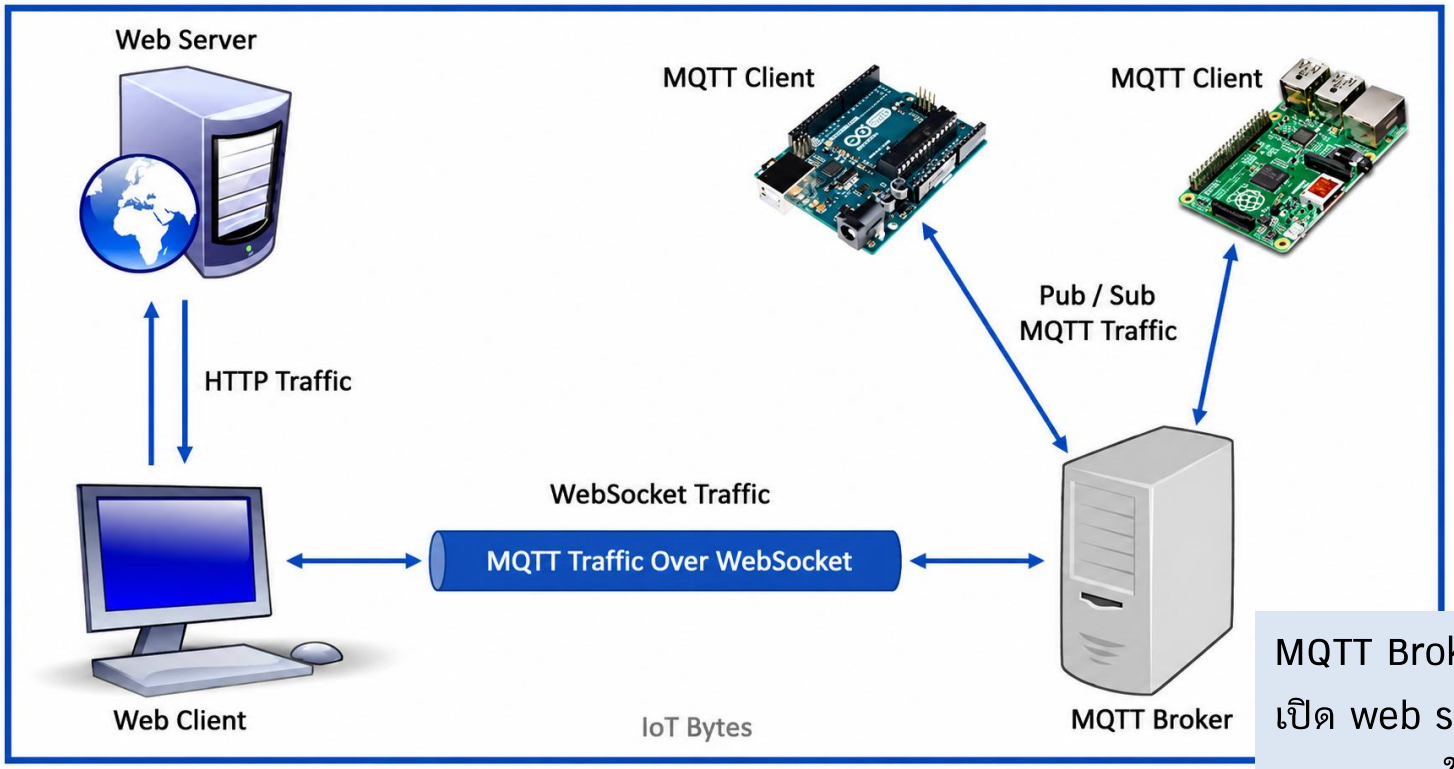
- ให้ Publisher (Raspi) ทำการส่งค่าข้อความใด ๆ ทุกๆ 2 วินาที โดยที่
  - เมื่อ Internet หลุด / ดับไป ให้ทำการแจ้งไปยัง LWT Subscriber ว่า Publisher OFFLINE

- ให้ Subscriber (Raspi) ทำการรับค่าข้อความ จาก Publisher โดยที่
  - เมื่อ Internet หลุด / ดับไป ให้ทำการแจ้งไปยัง LWT Subscriber ว่า Subscriber OFFLINE
  - เมื่อ Connect กลับเข้ามาใหม่ ก็ให้สามารถรับข้อความ ในช่วงที่หลุดหายไป ได้ครบถ้วนเช่นเดิม
  - ให้รอรับค่า เมื่อได้รับคำว่า ON ให้ LED ติด , เมื่อได้รับคำว่า OFF ให้ LED ดับ

    โดยที่ ให้ web browser ทำหน้าที่ Publisher ส่งค่า ON, OFF

WEB Socket

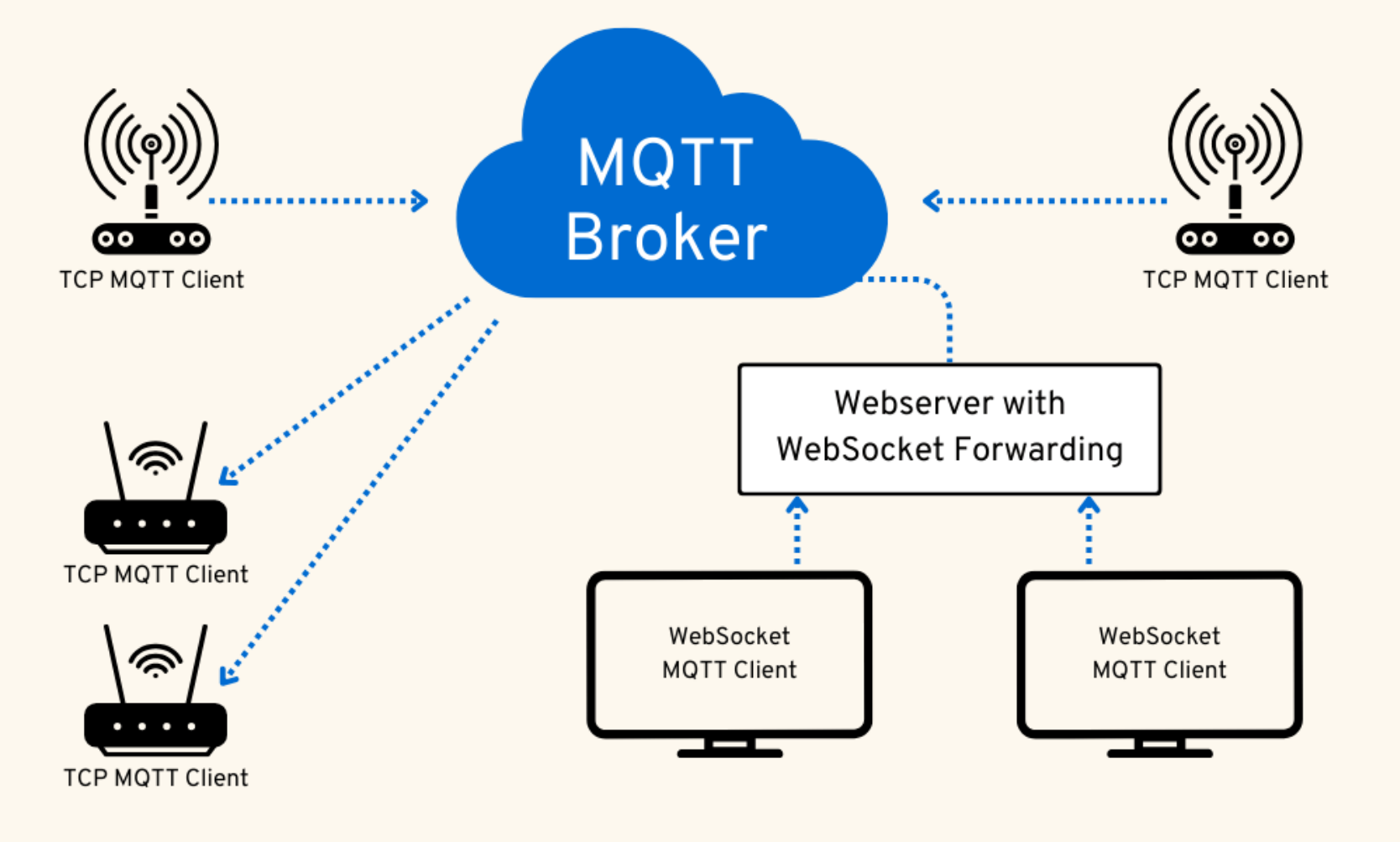
WebSocket คือ โปรโตคอลที่ใช้ในการสื่อสารแบบสองทาง (bidirectional) เปิดการเชื่อมต่อค้างอยู่ตลอดเวลา ทำให้สามารถส่งข้อมูลได้ real-time และลดความล่าช้าในการติดต่อสื่อสาร



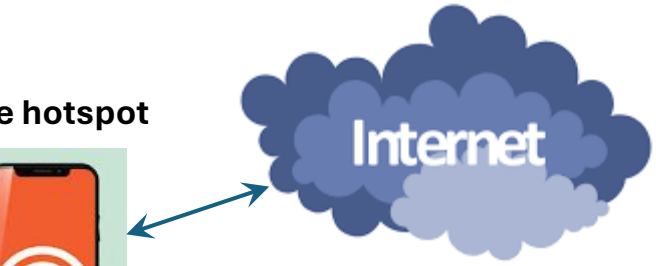
MQTT Broker ต้องเปิด web socket จึงจะสามารถใช้งานได้

MQTT over Websockets

MQTT over Websockets



Lab : Network Topology





mqtt-dashboard.com

หรือ

phycom.it.kmitl.ac.th

Port

mqtt : 1883

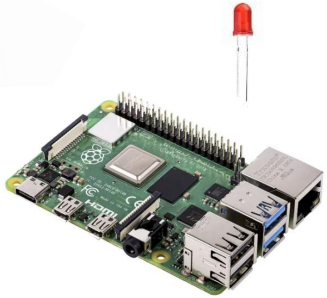
web socket : 8884

Mobile hotspot





Raspberry Pi
Publisher



Raspberry Pi
Subscriber

ต่อ ตัวต้านทาน 220 โอห์ม

และต่อหลอด LED



เปิด web browser

Run ไฟล์ control_rev1.html

ไฟล์ control_rev1.html แก้ broker, port, topic

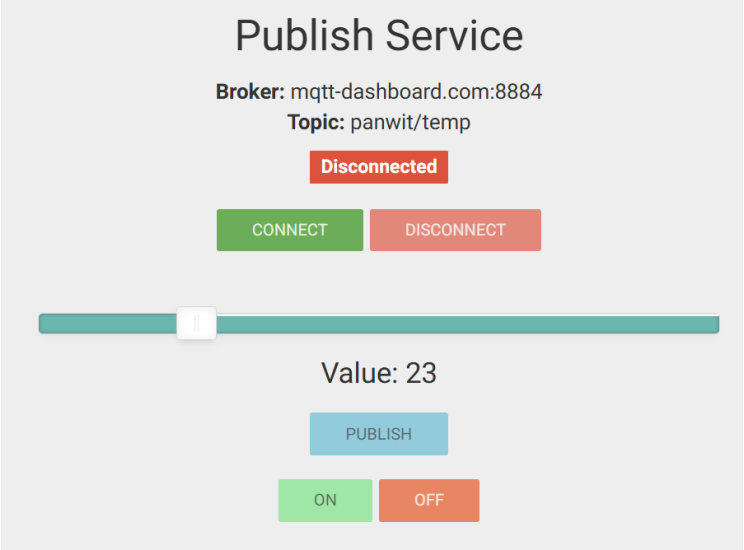
การทดลองที่ 4. MQTT Control via Websocket

1) เตรียมไฟล์

- แตกไฟล์ web.rar (Unzip) ไฟล์ทั้งหมดไว้ใน Folder เดียวกัน
- เปิดไฟล์ control_rev1.html

2) การควบคุมผ่าน MQTT

- ไฟล์ตัวอย่างทำหน้าที่ Publish ข้อมูลไปยัง Topic ที่กำหนด
  - เมื่อกดปุ่ม ON จะ publish ค่า on ไปยัง Broker
  - เมื่อกดปุ่ม OFF จะ publish ค่า off ไปยัง Broker
  - เมื่อ slide จะส่งค่าตามที่ Slide



การทดลองที่ 4. MQTT Control via Websocket

1. แก้ไขไฟล์ control_rev1.html ดังนี้

```
var MQTTbroker = 'mqtt-dashboard.com';   // broker
var MQTTport = 8884;                     // port ของ Webseocket ที่เปิดไว้
var MQTTsubTopic = 'temp';               //แก้ชื่อ topic ได้ตามใจชอบ
```

2. ทำการ Upload ไฟล์ ไปยัง web hosting (หรือ run ที่ client)

3. เขียนโปรแกรม ที่ Raspi เพื่อ Subscribe ค่าที่ส่งมาจากไฟล์ control_rev1.html
   - เมื่อกดปุ่ม ON ให้ LED ติด เมื่อกดปุ่ม OFF ให้ LED ดับ
   - เมื่อเลื่อน Slide bar ให้ LED หรี่สว่างตามค่าที่ Publish มา

แบบฝึกหัดที่ 2. MQTT Control via Websocket

1. แก้ไขไฟล์ control-rev1.html ให้

   ควบคุมการผสมสี Color LED เช่น

   1. Red : ON , Green : OFF , Blue : ON ให้ Color LED แสดงสีม่วง




THE END